



**Jose Said
Olano Garcia**

*Senior Software
Development Manager*

Sobre el presentador



josesaid@gmail.com



linkedin.com/in/josesaid



Spring · Java · Microservices · AWS

Acerca de esta presentación

Exploración completa de Spring AI 1.0 GA: fundamentos, ChatClient, RAG, Agents, Advisors, Memory, ETL Pipeline, Ollama, Observabilidad, MCP y Testing. Código Java real en cada slide.



Spring AI

*Generative AI para
el ecosistema Spring*

Spring AI 1.0 GA

Contenido

- ¿Qué es Spring AI?
- ChatClient & Chat Models
- Structured Output
- Modelos & Proveedores
- Advisors
- ETL Pipeline
- Observabilidad
- AI Testing
- Mejores Prácticas
- Arquitectura
- Prompt Templates
- RAG & Vector Stores
- AI Agents & Tool Calling
- Chat Memory
- Modelos Locales (Ollama)
- MCP Protocol
- Multimodalidad
- Conclusiones



¿Qué es Spring AI?

Propósito



Framework que simplifica el desarrollo de apps con IA generativa en el ecosistema Spring/Java.

Inspiración



Inspirado en LangChain y LlamaIndex (Python), pero diseñado nativamente para Java enterprise.

Portabilidad



Abstrae los proveedores de IA (OpenAI, Azure, Google...) con interfaces comunes e intercambiables.

Integración



Funciona nativamente con Spring Boot: autoconfiguración, `application.properties` y starters.



Arquitectura de Spring AI

Tu Aplicación Spring Boot



Spring AI Core · ChatClient · PromptTemplate · VectorStore · Advisors



Proveedores de IA · OpenAI · Azure · Vertex · Bedrock · Ollama...



ChatClient



Prompt
Template



Vector
Store



Advisors



Output
Parser



AI Client & Chat Model

ChatClient

API fluida y portátil para interactuar con cualquier LLM:

- Respuestas sincrónicas y streaming
- Mensajes de sistema, usuario y asistente
- Conversación multi-turno
- Advisors para pre/post-procesado
- Soporte multi-modal (texto + imagen)
- @Autowired listo con Spring Boot



```
@Autowired ChatClient chatClient;
```

```
// Respuesta simple
```

```
String r = chatClient
```

```
    .prompt()
```

```
    .user("What is Spring AI?")
```

```
    .call().content();
```

```
// Streaming
```

```
chatClient.prompt()
```

```
    .user("Explain step by step")
```

```
    .stream()
```

```
    .content()
```

```
    .subscribe(out::println);
```



Prompt Templates



PromptTemplate

Prompts reutilizables con variables {placeholder}. Separa lógica del texto del prompt.



System + User

Combina mensajes de sistema (rol/comportamiento) con mensajes del usuario.



Archivos .st

Templates cargados desde classpath como archivos StringTemplate — sin recompilar.

```
PromptTemplate pt = new PromptTemplate(
    "Translate {text} to {language}");
Prompt p = pt.create(Map.of("text","Hello","language","fr"));

// System + User
chatClient.prompt().system("You are a Java expert")
    .user("Explain records").call();
```



Structured Output

Spring AI convierte la respuesta del LLM en objetos Java tipados — sin parseo manual



```
// Define your Java Record
record Actor(String name, List<String> movies) {}

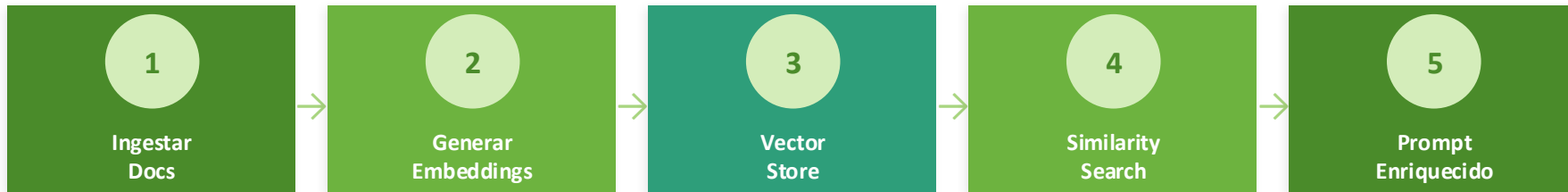
// Spring AI parses automatically
Actor actor = chatClient
    .prompt("Info about Keanu Reeves")
    .call()
    .entity(Actor.class); // no manual Jackson!

// Also: List<T> · Map<K,V> · BeanOutputConverter
```



RAG & Vector Stores

Retrieval Augmented Generation — Enriquece las respuestas con tu propia data



Vector Stores soportados:

PgVector

Redis

Chroma

Pinecone

Weaviate

Milvus

MongoDB Atlas

Azure AI Search

```
vectorStore.similaritySearch(SearchRequest.query("Java microservices").withTopK(5));
```




Modelos & Proveedores

Chat Models



GPT-4o, Claude 3.5, Gemini Pro, Llama 3, Mistral

Embedding Models



text-embedding-3, Titan Embeddings, Nomic, MiniLM

Image Generation



DALL-E 3, Stable Diffusion, Vertex Imagen

Speech & Audio



OpenAI Whisper, TTS — transcripción y síntesis

Todos los modelos comparten las mismas interfaces — cambiar de proveedor no requiere cambiar el código



AI Agents & Tool Calling

Tool Calling

Permite al LLM llamar funciones Java reales en tiempo de ejecución:

- Anota métodos con `@Tool` para exponerlos
- El modelo decide cuándo y cómo usarlos
- Herramientas estáticas y dinámicas
- Agentes que usan APIs, DBs y servicios
- Soporte para llamadas encadenadas



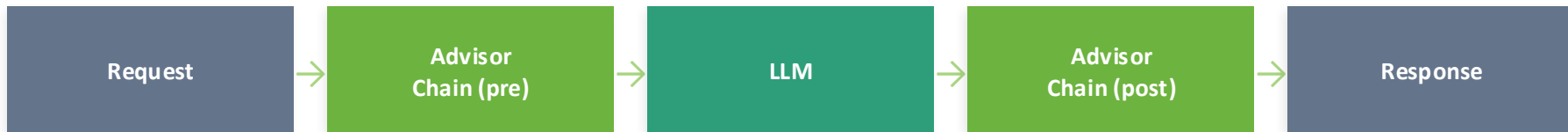
```
// Define a tool
@Tool(description =
    "Current stock price")
public double getPrice(String ticker) {
    return stockSvc.getPrice(ticker);
}

// Register in ChatClient
chatClient.prompt()
    .user("Analyze AAPL vs MSFT")
    .tools(new StockTools())
    .call().content();
```



Advisors & Interceptores

Interceptan y modifican el flujo de la conversación antes y después del LLM



MessageChatMemoryAdvisor

Inyecta automáticamente el historial en cada prompt — memoria transparente.



QuestionAnswerAdvisor

RAG automático: busca en VectorStore y enriquece el contexto del prompt.



Custom Advisor

Implementa CallAroundAdvisor para logging, seguridad, moderación o caché.

```
chatClient.prompt()  
  .advisors(new MessageChatMemoryAdvisor(mem), new QuestionAnswerAdvisor(vs))  
  .user(query).call().content();
```



Chat Memory



InMemoryChatMemory

Almacena historial en RAM. Ideal para desarrollo y PoC. Se pierde al reiniciar.



JdbcChatMemory

Persiste la conversación en base de datos relacional vía Spring JDBC. Listo para producción.



CassandraChatMemory

Almacenamiento distribuido en Apache Cassandra para alta disponibilidad.



```
// Configurar memoria persistente
ChatMemory memory = new JdbcChatMemory(jdbcTemplate);

// Advisor keeps history per conversationId
chatClient.prompt()
    .advisors(new MessageChatMemoryAdvisor(memory))
```

El historial se inyecta automáticamente en cada request — el LLM mantiene contexto sin código extra



ETL Pipeline para Documentos

Pipeline de tres fases para ingestar documentos en el Vector Store

1. Document Reader

Lee PDFs, HTML, Markdown, Word, JSON, CSV desde disco, URL o classpath.

2. Document Transformer

Divide, limpia y enriquece. `TokenTextSplitter`, `KeywordMetadataEnricher`...

3. Document Writer

Escribe los chunks vectorizados al Vector Store de tu elección.



```
// Reader → Transformer → Writer
new TokenTextSplitter()
    .apply(new PagePdfDocumentReader("docs.pdf"))
    .read()
    .forEach(vectorStore::add);

// DocumentReaders: PDF, HTML, Markdown, TIKA, GitHub, YouTube...
```



Modelos Locales con Ollama

Ejecuta LLMs localmente — sin APIs externas, sin costos por token, sin enviar datos a la nube



Privacidad Total

Los datos nunca salen de tu infraestructura. Ideal para datos sensibles o entornos regulados.



Cero Costos

Sin facturación por token. Perfecto para desarrollo, pruebas y batch processing.



Misma API

El código Java es idéntico. Solo cambia el starter y las properties — cero refactor.



```
// application.properties  
spring.ai.ollama.base-url=http://localhost:11434  
spring.ai.ollama.chat.options.model=llama3
```

```
// ChatClient works exactly like with OpenAI
```

```
String r = chatClient.prompt().user("Hello!").call().content();
```

Modelos: Llama 3, Mistral, Phi-3, Gemma 2, CodeLlama, Qwen, DeepSeek...



Observabilidad



Micrometer Metrics

Latencia, tokens usados y errores exportados a Prometheus, DataDog, CloudWatch...



Distributed Tracing

Trazas con Micrometer Tracing (Zipkin, Jaeger) para el ciclo completo de cada request.



Spring Actuator

/actuator/health y /actuator/metrics para monitorear el estado de los modelos en producción.



```
// application.properties
management.endpoints.web.exposure.include=health,metrics,info
management.tracing.sampling.probability=1.0

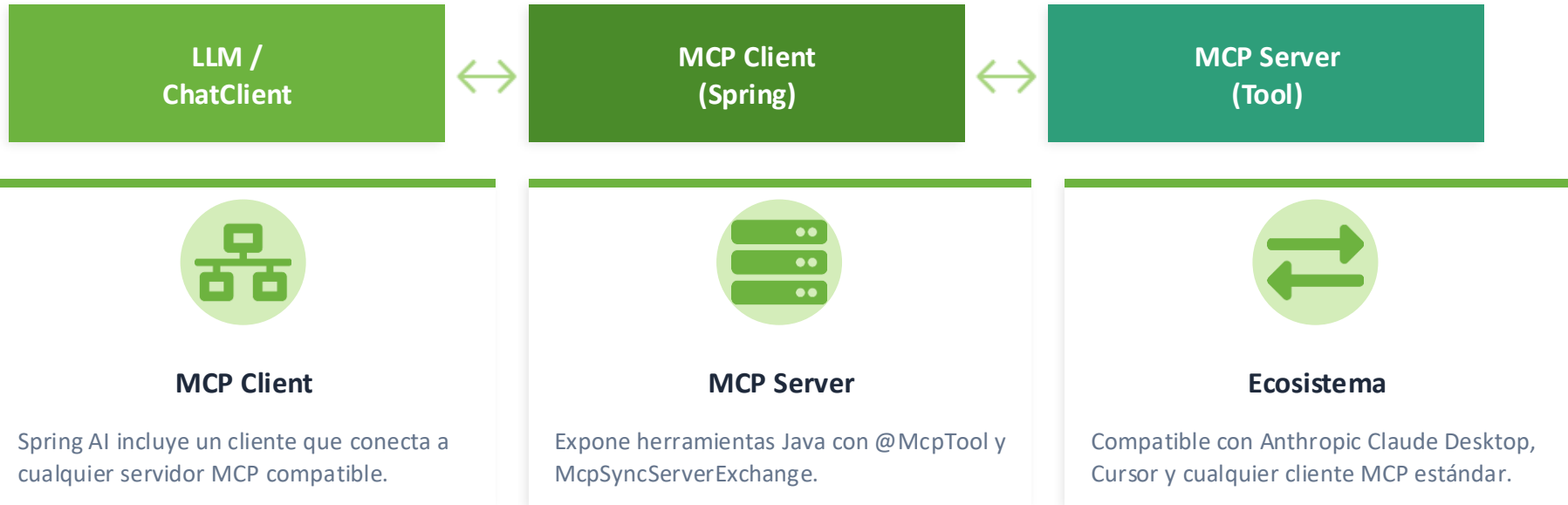
// Auto metrics available:
// spring.ai.chat.client.requests    (call counter)
```

Habilitado agregando spring-boot-starter-actuator + micrometer-registry-prometheus



Model Context Protocol (MCP)

Protocolo estándar para conectar LLMs con herramientas y fuentes de datos externas



```
spring.ai.mcp.client.sse.connections.myServer.url=http://localhost:8080/mcp
```




AI Testing & Evaluación

Spring AI incluye utilidades para evaluar la calidad y relevancia de las respuestas



RelevancyEvaluator

Evalúa si la respuesta es relevante a la pregunta usando el LLM como juez.



FactCheckingEvaluator

Verifica que la respuesta esté fundamentada en el contexto (anti-hallucination).



MockChatClient

Simula respuestas del LLM en tests unitarios sin llamadas reales a APIs externas.



```
@SpringBootTest
```

```
class AiTest {
```

```
    @Autowired RelevancyEvaluator evaluator;
```

```
    @Test void testRelevancy() {
```

```
        var req = new EvaluationRequest("Capital of Mexico?",
```

```
            List.of(), "Mexico City");
```

Multimodalidad

Envía texto, imágenes, audio y documentos al modelo — todo desde el mismo ChatClient



Visión (Imagen)

Analiza imágenes, capturas y diagramas.
Compatible con GPT-4o Vision, Claude 3.5,
Gemini.



Audio & Voz

Transcripción (Speech-to-Text) y síntesis
(TTS) con OpenAI Whisper y Azure Speech.



Documentos

Adjunta PDFs y archivos como contexto
directo en el prompt para análisis en línea.



```
// Analyze an image with GPT-4o Vision
var media = new Media(MimeTypeUtils.IMAGE_PNG,
    new ClassPathResource("diagram.png"));
String result = chatClient.prompt()
    .user(u -> u.text("What does this diagram show?")
        .media(media))
```



Mejores Prácticas



Externaliza prompts

Guarda templates en archivos .st del classpath.
Iteración sin recompilar.



Habilita Observabilidad

Configura Micrometer desde el inicio.
Monitorea latencia y tokens en cada entorno.



Usa ChatMemory

JdbcChatMemory en producción. InMemory solo para desarrollo y tests.



Ollama en desarrollo

Sin costos ni latencia de red. Cambia a cloud en producción con cero refactor.



Integra AI Testing

RelevancyEvaluator en CI/CD para detectar regresiones en calidad de respuestas.



Diseña para portabilidad

Injecta ChatClient vía Spring. Nunca acoplar código al SDK de un proveedor.



Conclusiones

Spring AI 1.0 GA

docs.spring.io/spring-ai

github.com/spring-projects/spring-ai



Portabilidad sobre +15 proveedores de IA



Integración nativa con Spring Boot



Chat, RAG, Agents, Memory, Multimodal



Modelos locales con Ollama — cero refactor



Observabilidad con Micrometer desde el inicio



Evaluadores para testing anti-hallucination



MCP: integración con cualquier tool externo



De PoC a producción sin reescribir código